

Digilence Program

AWS Lab Assignments

Three real world projects to build from scratch

Lab 01: EC2 + CloudWatch + SNS + Auto Scaling

Lab 02: Serverless Web App — Lambda + API Gateway + S3

Lab 03: High Availability — CloudFront + ALB + IAM

Before You Start

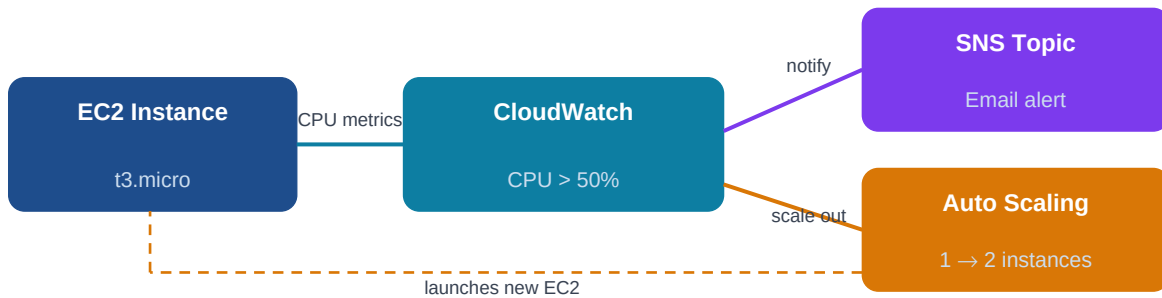
AWS account Free Tier | Region: us-east-1 | Key Pair downloaded | AWS CLI configured

Code files provided: [index.html](#) · [lambda_function.py](#)

■ ~35 minutes

■ Free Tier — EC2 t3.micro | SNS email free

Architecture Diagram



What to Build

- A Launch Template: t3.micro + httpd User Data showing instance ID on port 80
- An Auto Scaling Group using that template — Min: 1 | Desired: 1 | Max: 2
- An SNS Topic with your email subscribed and confirmed
- A CloudWatch Alarm: CPUUtilization > 50% — triggers SNS email AND ASG scale-out
- Run a CPU stress test and verify auto scale from 1 to 2 instances

Requirements

- Launch Template: AMI = Amazon Linux 2023 | t3.micro | SG = HTTP 80 + SSH 22
- User Data must write instance ID + AZ to index.html using instance metadata service
- ASG must be attached to a Target Group | Health check type = ELB
- SNS Topic type = Standard | Email subscription confirmed
- Alarm: Metric = CPUUtilization | Statistic = Average | Period = 1 min | Threshold > 50
- Alarm actions: SNS notification AND ASG scale-out policy
- Stress command: `sudo stress --cpu 4 --timeout 300s`

Deliverable — What Proves it Works

- ✓ Web page shows real instance ID (i-xxxxxxx) fetched from metadata

✓ Email notification received when alarm state changes to ALARM

✓ Second EC2 instance launches automatically — visible in ASG Instance Management tab

✓ After stress stops — alarm returns to OK and second instance terminates

■ Hint: Install stress with: `sudo yum install -y stress`. Keep the CloudWatch alarm open in a second browser tab to watch the state change live.

■ **Delete the ALB and ASG immediately after finishing — they charge per hour.**

■

■ Cleanup — Delete in This Order

→ ASG → Delete Auto Scaling Group (terminates all instances automatically)

→ EC2 → Delete Target Group | Delete Launch Template

→ SNS → Delete Topic and Subscription

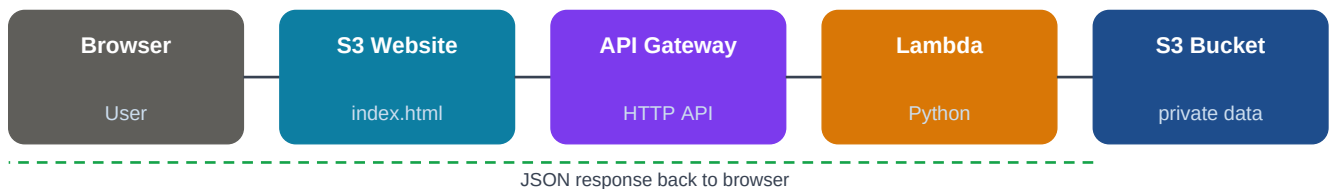
→ CloudWatch → Delete Alarm

■ ~40 minutes

■ Lambda: 1M free/month | API GW HTTP: 1M free
| S3: Free Tier

Architecture Diagram

IAM Role: Lambda can read/write S3



What to Build

- An S3 bucket with Static Website Hosting — hosts the provided index.html
- A second private S3 bucket for file uploads (data bucket)
- A Lambda function (Python 3.12) with two routes: /hello and /presign
- An HTTP API Gateway connected to the Lambda function
- An IAM Role on Lambda allowing S3 read/write on the data bucket only
- Update index.html with your API URL, upload to the website bucket, test everything

Requirements

- Website bucket: Static hosting enabled | Block Public Access OFF | public-read Bucket Policy
- Data bucket: Block Public Access ON (private) | CORS enabled for PUT
- Lambda: Runtime Python 3.12 | Env variable BUCKET_NAME = data bucket name
- Lambda IAM Role: s3:GetObject + s3:PutObject on data bucket only
- API Gateway: HTTP API | Routes: GET /hello GET /presign | CORS enabled | Allow-Origin: *
- In index.html: replace API_URL constant with your real API Gateway URL before uploading

Deliverable — What Proves it Works

- ✓ S3 website URL opens the web app in browser without AWS login

- ✓ "Call API" button returns JSON from Lambda with your name
- ✓ File upload via the web app stores file in the private S3 data bucket
- ✓ Direct S3 object URL of the uploaded file gives 403 Forbidden
- ✓ CloudWatch Logs shows Lambda invocations for each request

■ index.html — Line to Update

// REPLACE WITH YOUR API GATEWAY URL:

```
const API_URL = "https://YOUR-API-ID.execute-api.us-east-1.amazonaws.com/default";
```

■ lambda_function.py — Routes

```
def lambda_handler(event, context):  
    path = event.get("rawPath", "/")  
    if "/hello" in path:  
        name = params.get("name", "World")  
        return respond(200, {"message": f"Hello, {name}!"})  
    if "/presign" in path:  
        # returns S3 presigned URL for file upload  
    ...
```

- Hint: API Gateway CORS — set Allow-Origin: * | Allow-Methods: GET, POST, OPTIONS | Allow-Headers: Content-Type

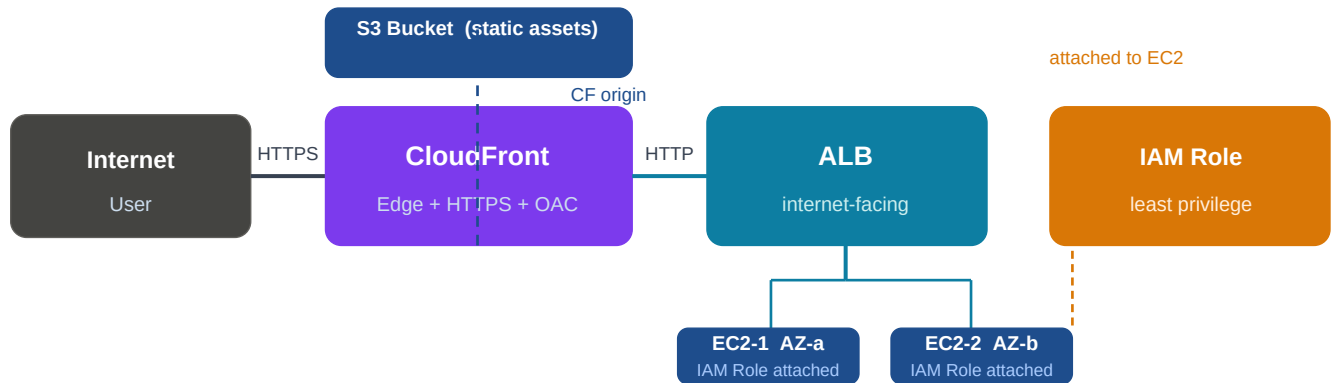
■ Cleanup — Delete in This Order

- Lambda → Delete function
- API Gateway → Delete HTTP API
- S3 → Empty and delete both buckets
- IAM → Delete Lambda execution role

■ ~40 minutes

■ CloudFront: 1 TB free/month | ALB: ~\$0.008/hr —
DELETE after lab

Architecture Diagram



What to Build

- Two EC2 web servers (different AZs) with httpd serving a page showing instance ID
- An IAM Role attached to both EC2s — least privilege (S3 read-only)
- A Security Group for EC2 that allows HTTP only from the ALB (not from internet directly)
- An Application Load Balancer across both AZs routing to both EC2s
- A CloudFront Distribution with the ALB as origin — HTTPS enabled
- Test that the CloudFront URL works and S3 + caching behaves correctly

Requirements

- EC2 instances: Amazon Linux 2023 | t3.micro | different AZs | User Data installs httpd + shows instance ID
- IAM Role: allows only s3:GetObject + s3:ListBucket — attach to both EC2s as Instance Profile
- EC2 Security Group inbound: HTTP port 80 from ALB-SG only (not 0.0.0.0/0)
- ALB: internet-facing | HTTP:80 listener | Target Group with both EC2s | Health Check path: /
- CloudFront: Origin = ALB DNS name | Origin Protocol = HTTP only | Viewer Protocol = Redirect to HTTPS
- CloudFront: Default TTL = 60 seconds | Price Class 100 (North America + Europe)
- Verify: ALB URL works | CloudFront URL works with HTTPS | Direct EC2 port 80 is blocked from internet

→ Test cache: update the web page on EC2 → create CloudFront Invalidation (/*) → verify new content appears

Deliverable — What Proves it Works

- ✓ ALB DNS name shows web page — refreshing alternates between EC2-1 (AZ-a) and EC2-2 (AZ-b)
- ✓ CloudFront URL (https://xxxx.cloudfront.net) opens the web page with HTTPS
- ✓ Direct access to EC2 Public IP on port 80 is blocked — only ALB and CloudFront work
- ✓ After updating EC2 web page content — CloudFront Invalidation makes new content appear
- ✓ CloudFront Cache Statistics show Cache HIT on repeated requests
- ✓ IAM Role is visible on EC2 → Security → IAM Role — and limits access correctly

■ Hint: When creating the CloudFront distribution — select the ALB DNS name as origin. Do NOT use the EC2 public IPs. Set Viewer Protocol Policy to "Redirect HTTP to HTTPS" for HTTPS enforcement.

■ Cache Invalidation: CloudFront → your distribution → Invalidation tab → Create invalidation → Object paths: /* → Create. Wait ~1 minute → refresh URL → new content appears.

■ **ALB charges per hour and CloudFront charges per request after free tier. Delete both immediately after the lab.**

■ Cleanup — Delete in This Order

- 1. CloudFront → Distributions → Disable distribution first (wait ~5 min)
- 2. CloudFront → Delete distribution after it is disabled
- 3. EC2 → Load Balancers → Delete ALB
- 4. EC2 → Target Groups → Delete Target Group
- 5. EC2 → Terminate both EC2 instances
- 6. EC2 → Delete ALB-SG and EC2-SG
- 7. IAM → Delete the EC2 Instance Role and Instance Profile